Multivariate Time-Series Prediction Using Deep Learning - Appliance Energy Consumption Prediction

Name: Thiyara Gunawardena

Date: 03/31/2026

Table of Contents

1. Introduction 3
2. Data Insights 4
3. Preprocessing 7
4. Feature Engineering 9
5. Model Development 12
   - 5.1. Baseline Models 12
   - 5.2. Deep Learning Model Design 12
     - 5.2.1. Design Choices and Justification 13
6. Results 16
   - 6.1. Evaluation Metrics and Baseline Comparison 16
   - 6.2. Visualization of Model Performance 17
   - 6.3. Key Observations 20
7. Model Optimization 21
   - 7.1. Optimization Techniques Applied 21
   - 7.2. Improvement in Performance Metrics 23
   - 7.3. Key Observations 24
   - 7.4. Training Behavior After Optimization 24
8. Challenges and Solutions 25
9. Conclusion 27
10. References 28

1. Introduction

This assessment focuses on predicting appliance energy consumption using a multivariate time-series dataset. The dataset contains environmental, temporal, and energy-related features recorded at 10-minute intervals over several months.

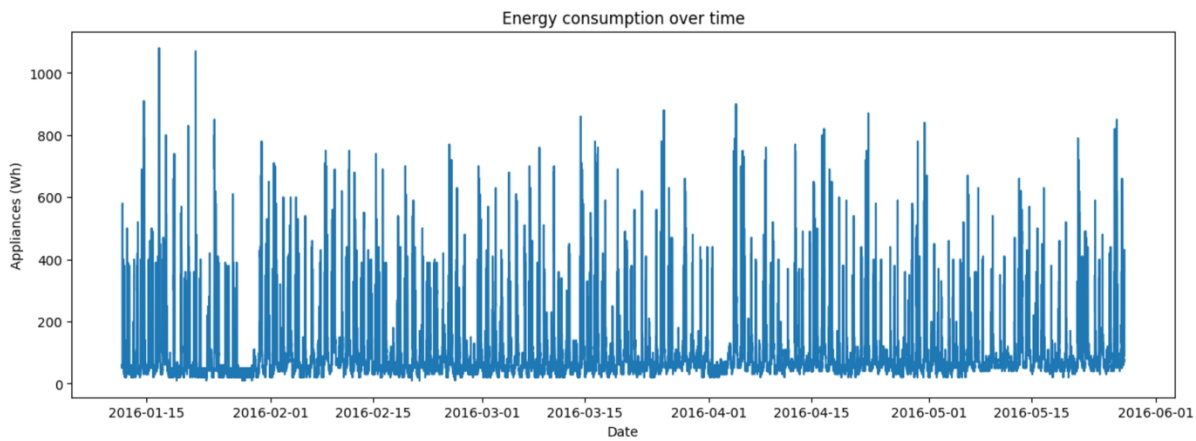
The objective of this project is to preprocess the dataset, engineer meaningful features, and develop a deep learning model to accurately predict energy consumption (Appliances).

The project follows a structured pipeline including data understanding, preprocessing, feature engineering, model development, evaluation, and optimization.

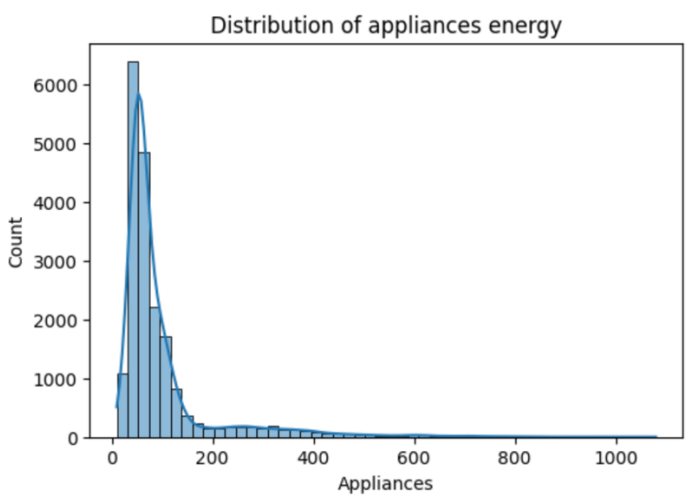
2. Data Insights

Exploratory Data Analysis (EDA) was conducted to understand patterns, relationships, and distributions within the dataset.

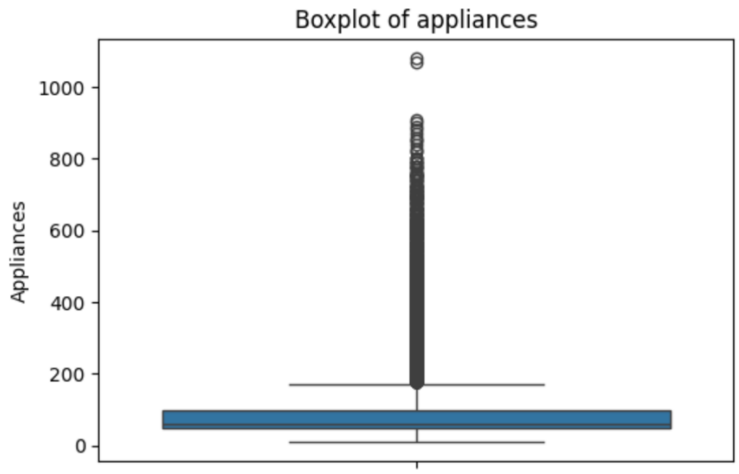
A time-series plot of appliance energy consumption revealed significant fluctuations over time, with frequent spikes in energy usage. This indicates that energy consumption is highly dynamic and influenced by temporal and environmental factors. While no clear long-term trend was observed, strong short-term variations were evident.



The distribution of the target variable (Appliances) was analyzed using a histogram and density plot. The results showed that the data is right-skewed, with the majority of values concentrated between 50 Wh and 150 Wh, and a small number of extreme values exceeding 1000 Wh. This confirms the presence of outliers and a non-normal distribution.



A boxplot further highlighted the presence of numerous extreme values. These outliers may negatively impact model performance and will be addressed during the preprocessing stage.



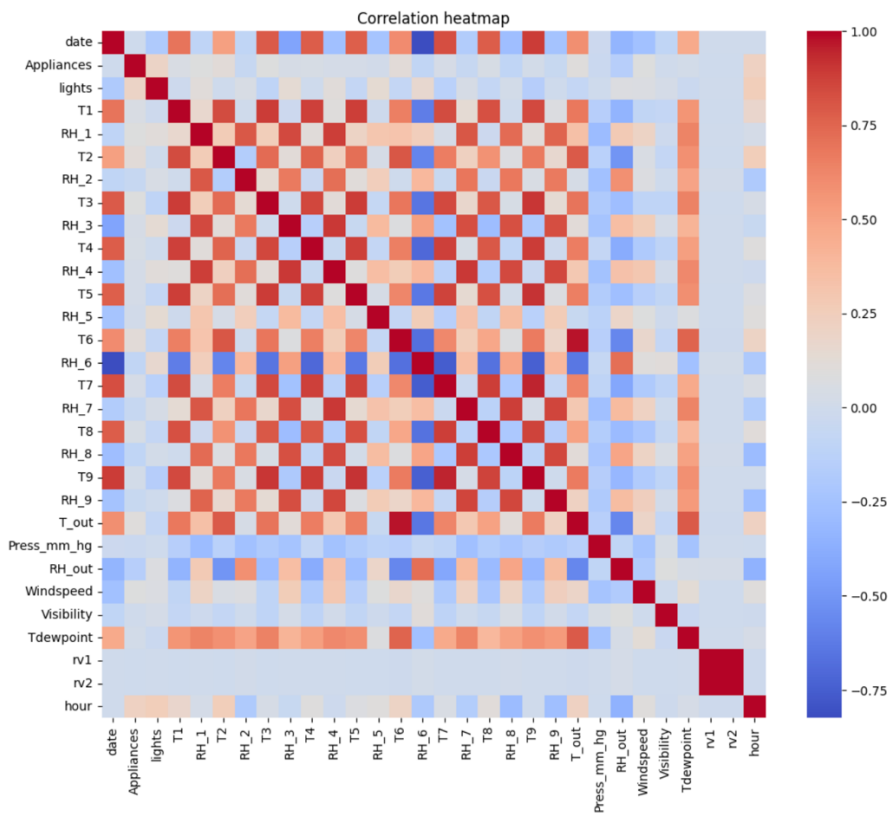
A correlation heatmap was generated to analyze relationships between features. The analysis revealed strong positive correlations among indoor temperature features (T1-T9) and humidity features (RH_1-RH_9), indicating redundancy and similar environmental conditions across different areas. This suggests that some features may be reduced or combined during feature selection.

The target variable (Appliances) showed weak linear correlation with most individual features, indicating that energy consumption is influenced by complex and nonlinear interactions rather than a single dominant variable.

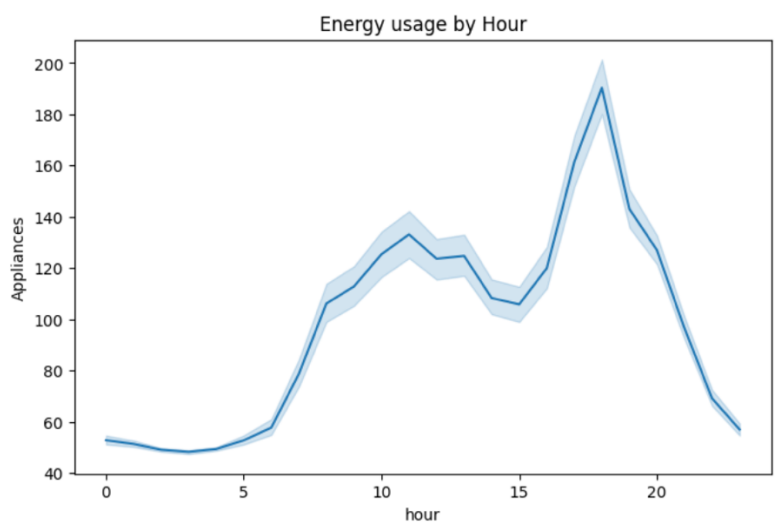
Outdoor variables such as temperature and dew point showed moderate correlation with indoor conditions, suggesting that external weather factors influence indoor environments.

The variables rv1 and rv2 exhibited no meaningful correlation with the target variable or other features but were highly correlated with each other, indicating that they are likely random noise variables. These features will be excluded in subsequent modeling steps due to their lack of predictive value.

Overall, the correlation analysis suggests the need for nonlinear modeling approaches such as deep learning to effectively capture the underlying relationships in the data.



Additionally, a time-based analysis revealed a clear daily pattern in energy consumption, with lower usage during nighttime hours and peak consumption occurring in the evening. This confirms a strong temporal dependency, highlighting the importance of incorporating time-based features into the model.



Overall, the EDA findings indicate that energy consumption is influenced by temporal patterns, nonlinear relationships, and the presence of outliers. These insights will guide the feature engineering and model development phases.

3. Preprocessing

Initial preprocessing steps were performed to prepare the dataset for further analysis and modeling.

The dataset was loaded using the Pandas library, and its structure was examined using summary functions such as shape, column names, and data types to understand the overall structure of the data.

The "date" column was converted from string format to datetime format to enable time-based operations and analysis. The dataset was then sorted based on the date column to maintain temporal consistency, which is essential for time-series modeling and to prevent data leakage.

A statistical summary of the dataset was generated to analyze the distribution of variables. It was observed that the target variable, "Appliances", exhibits a wide range of values, suggesting the presence of potential outliers.

Further analysis revealed that the energy consumption values are right-skewed, with a maximum value of 1080 Wh significantly higher than the median value of 60 Wh. This indicates the presence of extreme values, which may negatively impact model performance if not properly handled.

Additionally, environmental variables such as temperature and humidity show moderate variation, indicating their potential influence on energy consumption patterns. These observations will guide subsequent exploratory data analysis and feature engineering steps.

<u>Handling Missing Values</u>

A missing value analysis was conducted to identify incomplete data points across all features. The results showed that there are no missing values present in the dataset.

Since the dataset is complete, no imputation techniques (such as mean, median, or mode imputation) or record removal strategies were required. This ensures that no information loss or artificial bias is introduced during preprocessing.

<u>Outlier Detection and Treatment</u>

Outliers in the target variable (Appliances) were identified using the Interquartile Range (IQR) method. The lower and upper bounds were calculated based on the first (Q1) and third quartiles (Q3).

The analysis confirmed the presence of extreme values, as previously observed in the boxplot and distribution analysis.

Instead of removing these outliers, clipping was applied to limit extreme values within the defined bounds. This approach was chosen to preserve the temporal continuity of the dataset, which is critical for time-series modeling. Removing rows could disrupt sequential dependencies and negatively affect model performance.

Data Scaling and Normalization

Min-Max scaling was applied to normalize feature values between 0 and 1. This technique was selected because deep learning models, particularly LSTM networks, are sensitive to the scale of input data and perform better when features are normalized.

Separate scalers were used for input features and the target variable to ensure proper normalization and enable inverse scaling of predictions during model evaluation.

The scaling parameters were fitted only on the training dataset and applied to the test dataset to prevent data leakage and ensure realistic model evaluation.

Standardization (Z-score) was considered but not used, as it does not bound the output to a fixed range, which can slow down LSTM convergence.

Data Splitting

The dataset was split into training and testing sets using an 80:20 ratio. Since this is a time-series dataset, a chronological split was applied, where the first 80% of the data was used for training and the remaining 20% for testing.

After this split, the training sequences were further divided, with the last 10% of the training portion used as a validation set. This validation set was used to monitor model performance during training and to support early stopping and hyperparameter tuning.

This approach preserves temporal order and prevents data leakage, ensuring that future information is not used during model training.

4. Feature Engineering

Feature engineering was performed to enhance the predictive capability of the model by creating informative features and selecting the most relevant variables.

1. Time-Based Features

Time-based features such as hour, day of the month, month, and day of the week were extracted from the timestamp to capture temporal variations in energy consumption. These features are essential as energy usage patterns are strongly influenced by human routines and daily schedules. A binary indicator (is_weekend) was created to distinguish between weekdays and weekends, as occupancy patterns and appliance usage tend to differ significantly during weekends. Additionally, a categorical feature (time_of_day) was introduced by grouping hours into distinct periods (morning, afternoon, evening, and night). This allows the model to capture broader behavioral patterns beyond individual hours. Furthermore, specific peak-period indicators such as morning peak, evening peak, and nighttime were generated to represent typical high and low activity periods. These features help the model better understand variations in energy consumption driven by daily human behavior.

2. Lagged Features

Lagged features were created using past energy consumption values to capture temporal dependencies in the time-series data. Specifically, lag values at 10 minutes (lag_1), 30 minutes (lag_3), 1 hour (lag_6), 2 hours (lag_12), and 24 hours (lag_144) were generated. To determine appropriate lag intervals, an autocorrelation analysis was conducted. The autocorrelation plot shows a strong positive correlation at small lags, indicating that recent energy consumption values have a significant influence on current consumption. The correlation gradually decreases as the lag increases, confirming that the impact of older observations diminishes over time.

Additionally, the presence of a slow decay pattern and periodic structure in the autocorrelation plot suggests recurring daily behavior in energy usage. Based on these observations, both short-term lags (to capture immediate dependencies) and long-term lags (to capture daily cycles) were selected to effectively model temporal patterns in the dataset.

3. Rolling Averages and Moving Window Features

Rolling window statistics were computed to capture local trends and smooth short-term fluctuations in energy consumption. Specifically, rolling mean (1-hour and 3-hour windows) and rolling standard deviation (1-hour window) were calculated.

To prevent data leakage, a shift operation was applied before computing rolling statistics, ensuring that only past values were used in feature construction. The rolling mean features help capture underlying trends and smoothed consumption patterns, while the rolling standard deviation captures short-term variability and fluctuations in energy usage. These features improve the model's ability to learn both stable patterns and dynamic changes in energy consumption over time.

4. Interaction Features

Interaction features were generated by combining relevant variables to capture nonlinear relationships. These include interactions between temperature and humidity (e.g., T1 * RH_1 and T_out * RH_out), as environmental conditions often influence energy consumption jointly. Additionally, an interaction between lighting usage and time (lights * hour) was created to capture how lighting behavior varies across different times of the day. This helps model behavioral patterns where lighting usage is more prominent during certain hours, such as evenings or low-light conditions.

5. Feature Selection

Feature selection was performed to identify the most relevant variables and improve model performance while reducing dimensionality and overfitting. A combination of correlation analysis and Recursive Feature Elimination (RFE) was used.

Although rolling window features were generated to capture short-term trends and smooth fluctuations, they were not selected in the final feature set. This is because rolling features are derived from lagged values and therefore introduce redundant information. The feature selection process identified lag-based features as more directly informative, especially given the strong autocorrelation observed in the dataset. As a result, rolling features did not significantly improve predictive performance and were excluded to maintain model simplicity.

- Correlation Analysis

Correlation analysis was conducted on the training dataset to evaluate the relationship between each feature and the target variable (Appliances). Features with higher absolute correlation were selected, as they have stronger predictive power. The results showed that lag-based features, particularly lag_1, had the highest correlation, confirming the strong temporal dependency in energy consumption. Although rolling features also showed relatively high correlation, they were later excluded due to redundancy.

- Recursive Feature Elimination (RFE)

RFE was applied using a Linear Regression model to further refine the feature set. It iteratively removes less important features and retains the most significant ones based on model performance. This helped eliminate redundant and less informative features while improving model efficiency.

- Final Selected Features

The final selected features were:

- Lag feature: lag_1
- Time-based features: hour, is_night, time_of_day
- Interaction feature: lights
- Environmental features: T1, T2, T3, T6, T_indoor_avg

These features represent a combination of temporal dependencies, daily patterns, and environmental conditions influencing energy consumption.

Feature selection was performed only on the training data to avoid data leakage and ensure reliable model evaluation.

5. Model Design

5.1. Baseline Models

To establish a reference for model performance, two baseline models were implemented: Linear Regression and Random Forest Regressor.

Linear Regression was selected as a simple and interpretable model capable of capturing linear relationships between input features and energy consumption. Random Forest, a tree-based ensemble method, was included to model potential non-linear interactions within the data.

Preliminary evaluation indicated that Linear Regression outperformed Random Forest, suggesting that the engineered features, including lag variables and time-based features, effectively capture much of the underlying structure in the dataset.

Therefore, Linear Regression was selected as the primary baseline, and subsequent deep learning models are evaluated against this benchmark.

5.2. Deep Learning Model Design

To model temporal dependencies in appliance energy consumption, deep learning models were designed using sequence-based inputs. The dataset was transformed into time-series sequences with a lookback window of 12-time steps (equivalent to 2 hours of historical data). Each input sample has the shape *(12, n_features)*, allowing the models to learn temporal patterns effectively.

Four different deep learning architectures were implemented to capture varying levels of complexity in the data.

1. LSTM_Model_A (Baseline LSTM Model)

The first model is a simple sequential architecture consisting of:

- LSTM layer (32 units)
- Dense output layer (1 unit)

This model captures basic temporal dependencies and serves as the baseline deep learning model for comparison.

2. LSTM_Model_B (Enhanced LSTM Model)

This model extends the baseline by increasing representational capacity:

- LSTM layer (64 units)
- Dropout layer (rate = 0.1)
- Dense hidden layer (32 units, ReLU activation)

- Dense output layer (1 unit)

Compared with the baseline model, this architecture provides a richer feature representation while remaining relatively simple and computationally efficient. The LSTM layer captures temporal dependencies in the 12-step input sequence, the dropout layer helps reduce overfitting, and the additional dense hidden layer supports non-linear learning from the extracted temporal patterns. Among the initial deep learning models, LSTM_Model_B achieved the most balanced performance and showed stable convergence behavior, making it the selected model for further hyperparameter optimization.

3. LSTM_Model_C (CNN-BiLSTM-Attention Model)

A hybrid architecture was developed to capture both local and long-range dependencies:

- Conv1D layer (32 filters, kernel size = 2, ReLU activation)
  -> Extracts short-term temporal patterns
- Dropout layer
- Bidirectional LSTM layer (64 units)
  -> Captures forward and backward temporal dependencies
- Attention mechanism, implemented using:
  - Dense layer with Softmax activation
  - RepeatVector, Permute, and Multiply layers
    -> Allows the model to focus on important time steps
- Dense layers (32 -> 1) for final prediction

This architecture is designed for more complex sequence modeling and improved interpretability.

4. GRU_Model_D (GRU-Based Model)

A GRU-based architecture was implemented as a computationally efficient alternative to LSTM:

- GRU layers (stacked based on configuration)
- Dropout layers
- Dense layers (32 -> 1)

GRU simplifies the gating mechanism compared to LSTM while maintaining strong performance for sequence learning.

5.2.1. Design Choices and Justification

1. Use of LSTM and GRU Layers

LSTM and GRU are specifically designed for time-series data and were selected to:

- Capture temporal dependencies

- Handle sequential patterns and lag effects
- Model non-linear relationships in energy consumption

2. Use of Conv1D Layer (Hybrid Model)

The Conv1D layer was used in the hybrid architecture (Model C) to:

- Capture short-term local trends
- Extract features before sequence modeling
- Improve representation learning

This improves feature representation and enhances overall model performance.

3. Attention Mechanism (Model C)

The attention mechanism enables the hybrid model to:

- Assign importance to different time steps
- Focus on critical periods influencing predictions

This improves both interpretability and predictive capability.

4. Activation Functions

- ReLU (Rectified Linear Unit) used in Dense layers:
    - Prevents vanishing gradient problem
    - Enables faster and more stable convergence
- Tanh activation used internally in LSTM/GRU:
    - Suitable for sequential data
    - Produces bounded outputs for stable learning
- Softmax (Used in Attention layer):
    - Generates normalized attention weights

5. Loss Function: Mean Absolute Error (MAE)

MAE was selected because:

- It provides a direct and interpretable measure of prediction error (in Wh)
- It is less sensitive to extreme outliers compared to MSE

6. Optimizer: Adam

The Adam optimizer was used due to:

- Adaptive learning rate capability
- Faster convergence compared to traditional optimizers

- Robust performance across different architectures

7. Sequence Length (Lookback Window)

A sequence-based approach was used to model temporal dependencies. Initially, multiple lookback values (6,12, 24, and 48) were experimented.

The final model uses a lookback window of 12-time steps (2 hours) as it provided the best balance between:

- Capturing short-term temporal dependencies
- Avoiding unnecessary model complexity
- Maintaining stable and efficient training

6. Results

6.1. Evaluation Metrics and Baseline Comparison

To evaluate model performance, four standard regression metrics were used:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Percentage Error (MAPE)
- Coefficient of Determination (R^2)

Baseline models (Linear Regression and Random Forest) were first evaluated to establish reference performance.

Baseline Model Performance

The Linear Regression model outperformed Random Forest across all metrics, indicating that simpler linear relationships still capture a significant portion of the data variance. In contrast, Random Forest showed poor generalization, likely due to its inability to effectively model temporal dependencies.

Model	**MAE (Wh**	**RMSE (Wh)**	**MAPE (%)**	**R^2**
Linear Regression	14.15	21.70	19.35	0.6813
Random Forest	21.72	32.17	28.49	0.3054

Deep Learning Model Performance

	Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	MAE (Z)	RMSE (Z)	R2
0	LSTM_Model_A	13.0876	21.7601	16.3928	0.2981	0.4956	0.6822
1	LSTM_Model_B	12.4266	21.7288	15.8039	0.2830	0.4949	0.6831
2	LSTM_Model_C	19.0102	29.9137	20.0914	0.4330	0.6814	0.3994
3	GRU_Model_D	15.6340	24.2058	17.6772	0.3561	0.5513	0.6067

Among all deep learning models, LSTM_Model_B achieved the best performance, with:

- Lowest MAE (12.42 Wh)
- Lowest RMSE (21.72 Wh)
- Lowest MAPE (15.8%)

- Highest R^2 (0.68)

This indicates that adding a dense layer and dropout improved generalization compared to the simpler LSTM_Model_A.

Comparison with Baseline Models

- LSTM_Model_B outperformed Linear Regression in all key metrics:
  - MAE improved from 14.15 -> 12.42
  - MAPE reduced from 19.35% -> 15.80%
  - R^2 increased from 0.6813 -> 0.6831

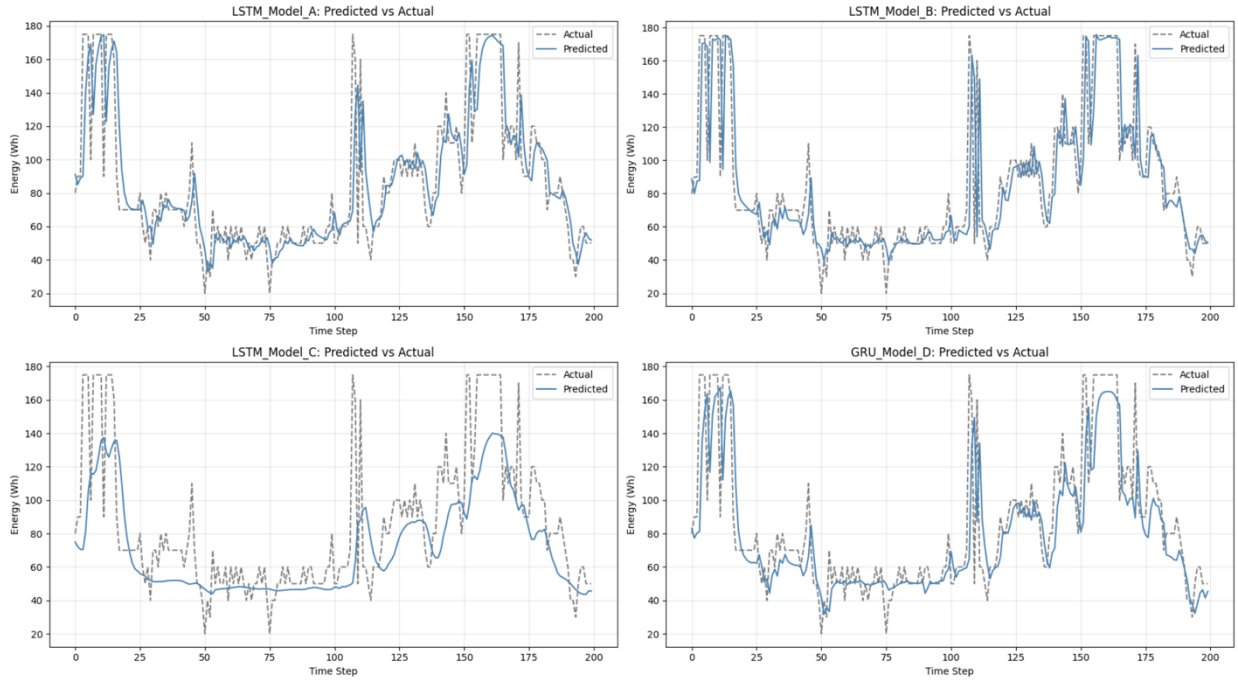
Random Forest performed significantly worse across all metrics, suggesting that tree-based models struggle to capture temporal dependencies in this dataset. This confirms that sequence-based deep learning models particularly LSTM models are more effective for this time-series prediction task.

6.2. Visualization of Model Performance

Predicted vs. Actual Values

The predicted vs. actual plots show that:

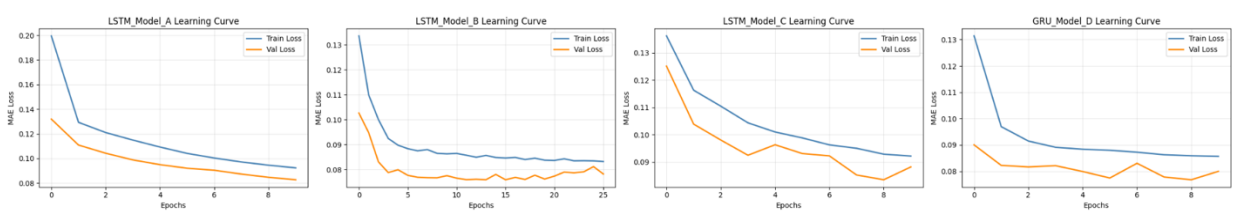
- LSTM_Model_B closely follows the true energy consumption trends
- Peak patterns and sudden changes are captured more effectively than in other models
- LSTM_Model_C shows noticeable deviations, suggesting potential overfitting due to increased model complexity



Learning Curves (Training vs Validation Loss)

The training history plots indicate:

- Rapid convergence within early epochs
- LSTM_Model_B achieves lower validation loss compared to other models, indicating improved predictive performance
- Slight fluctuations are observed in the validation loss of LSTM_Model_B, likely due to increased model complexity
- LSTM_Model_A shows smoother but higher validation loss, indicating more stable but less accurate learning
- No significant overfitting is observed across models due to the use of dropout and regularization

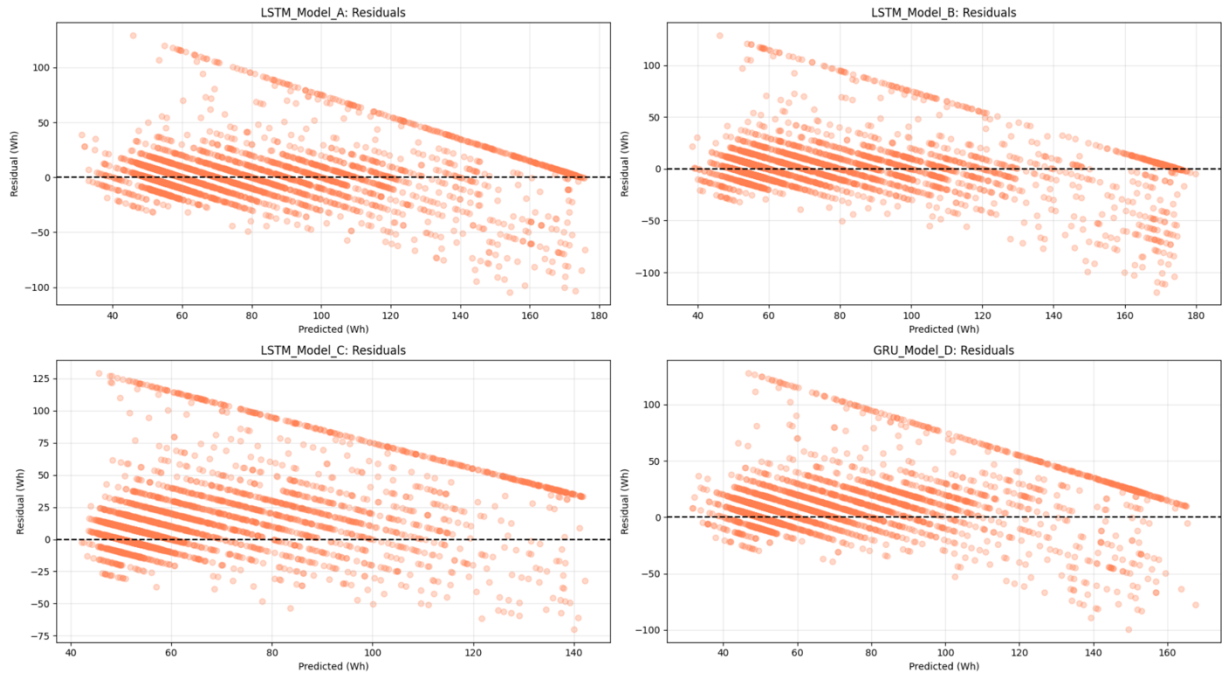


Residual Analysis

Residual plots show:

- Errors are centered around zero for most predictions
- Slight increase in spread at higher predicted values, indicating minor variance instability

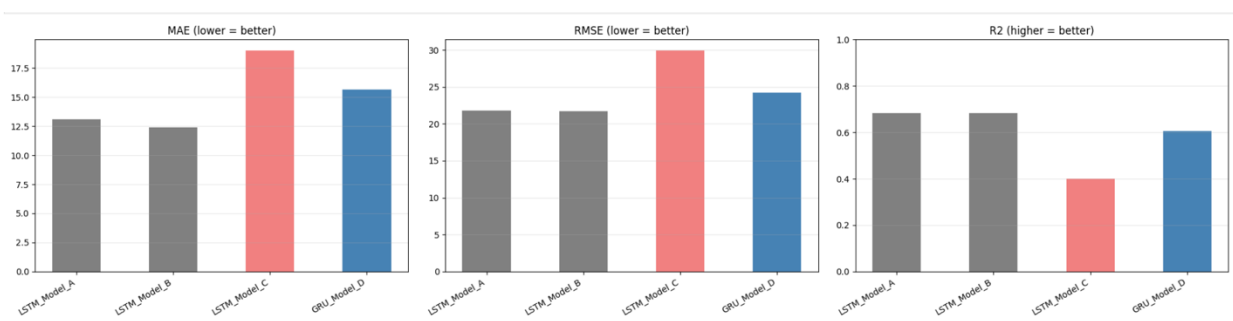
- A funnel-shaped pattern suggests heteroscedasticity (variance increases with prediction magnitude)
- LSTM_Model_B exhibits tighter residual distribution compared to other models



<u>Metric Comparison Plots</u>

Bar chart comparisons confirm that:

- LSTM_Model_B achieves the lowest overall error across most evaluation metrics
- LSTM_Model_C performs worst due to excessive complexity and overfitting
- GRU_Model_D performs moderately but does not surpass LSTM variants



6.3. Key Observations

- Sequence-based deep learning models outperform traditional machine learning models by effectively capturing temporal dependencies
- Moderate model complexity (LSTM_Model_B) performs better than overly complex architectures
- Incorporating temporal sequences significantly improves predictive accuracy
- The attention-based model (LSTM_Model_C) underperformed due to increased complexity and limited data support

7. Model Optimization

7.1. Optimization Techniques Applied

To improve predictive performance and generalization, a structured optimization process was applied to the best-performing baseline model, LSTM_Model_B**.** Multiple techniques were explored to balance accuracy, stability, and model complexity.

Hyperparameter Tuning

A randomized search strategy was employed to explore different combinations of hyperparameters efficiently. Instead of exhaustive grid search, a subset of configurations was sampled to reduce computational cost while maintaining diversity.

The following hyperparameters were tuned:

- Number of LSTM units: {64, 128}
- Number of recurrent layers: {1, 2}
- Learning rate: {0.001, 0.0005}
- Dropout rate: {0.1, 0.2, 0.3}
- Batch size: {16, 32}

Each configuration was evaluated based on validation MAE and RMSE, ensuring both average and large-error performance were considered.

Sequence Length Optimization (Lookback Tuning)

Different sequence lengths (lookback values) were experimentally evaluated:

- Lookback = 12 (2 hours)
- Lookback = 24 (4 hours)
- Lookback = 48 (8 hours)

Results showed that:

- Increasing lookback did not consistently improve performance
- Larger lookbacks introduced noise and reduced generalization
- A lookback of 12 provided the best balance between capturing temporal dependencies and model stability

Loss Function Exploration

Different loss functions were tested:

- Mean Absolute Error (MAE)

- Huber Loss

Findings:

- Huber loss improved robustness to outliers
- However, it did not consistently outperform MAE in final evaluation
- MAE was retained as it provided more stable and interpretable results

Early Stopping

To prevent overfitting and unnecessary training, EarlyStopping was applied with:

- Patience = 35 epochs
- Monitoring validation loss
- Restoring best model weights

This ensured that training stopped when the model stopped improving, improving efficiency and preventing overfitting.

Regularization Techniques

To prevent overfitting and improve generalization:

- Dropout layers (0.1 - 0.3) were applied
- Higher dropout improved generalization but slightly increased error variance
- L2 regularization was experimented with in deeper configurations
- Model complexity was controlled (avoiding unnecessary layers)

Learning Rate Scheduling

A ReduceLROnPlateau strategy was used to dynamically adjust the learning rate:

- Reduction factor = 0.3
- Triggered when validation loss plateaus

This helps the model converge more smoothly and avoid oscillations

Final Optimized Configuration

The best-performing hyperparameter combination was:

- LSTM units: 64
- Number of layers: 1
- Learning rate: 0.0005
- Dropout rate: 0.3
- Batch size: 32

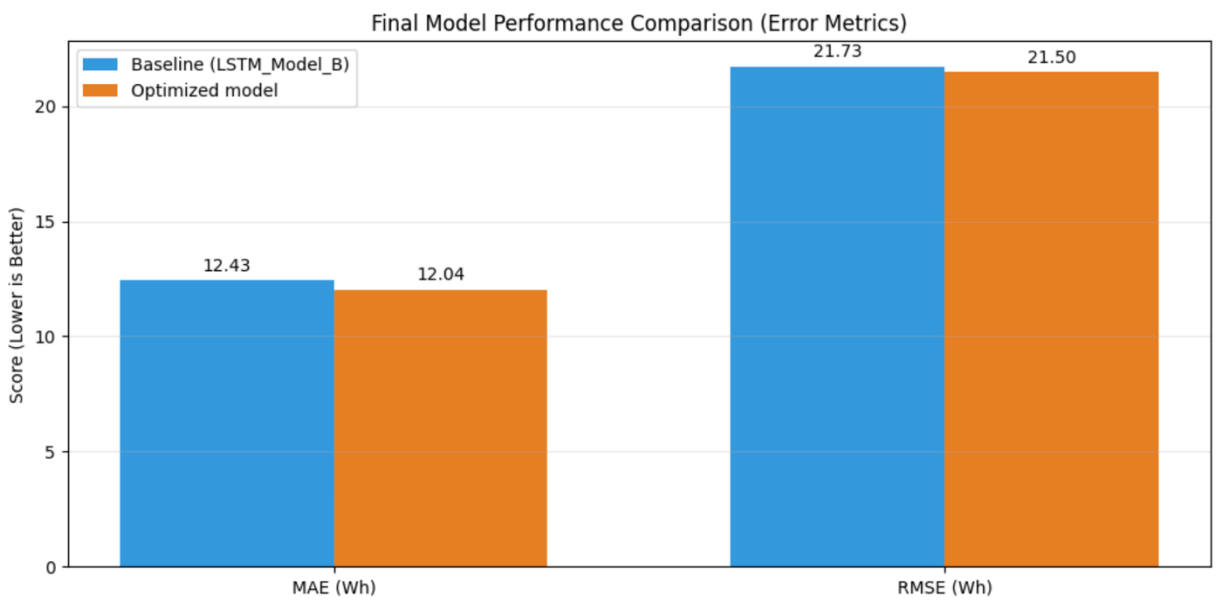
This configuration provided the best balance between accuracy, stability and generalization.

7.2. Improvement in Performance Metrics

The optimized LSTM_Model_B demonstrates consistent improvements across all evaluation metrics compared to the baseline model.

Performance Comparison

	Stage	MAE (Wh)	RMSE (Wh)	MAPE (%)	R2
0	Baseline (LSTM_Model_B)	12.4266	21.7288	15.8039	0.6831
1	Optimized model	12.0368	21.4959	15.5272	0.6899



Analysis of Improvements

• MAE decreased from 12.43 Wh to 12.04 Wh, indicating better average prediction accuracy
• RMSE decreased from 21.73 Wh to 21.50 Wh, showing improved handling of larger errors
• MAPE reduced from 15.80% to 15.53%, reflecting more stable percentage-based predictions
• R^2 increased from 0.6831 to 0.6899, indicating improved variance explanation

These results confirm that the optimization process successfully enhanced both the accuracy and generalization capability of the model without introducing trade-offs. Unlike earlier optimization attempts that introduced trade-offs between metrics, the final configuration achieved

simultaneous improvements across all evaluation metrics, indicating a well-balanced and robust model.

7.3. Key Observations

• Hyperparameter tuning improved both predictive accuracy and generalization capability of the model

• Learning rate reduction and early stopping contributed to stable and efficient convergence

• Appropriate dropout levels helped prevent overfitting while maintaining model performance

• The optimized model achieved improved variance explanation (higher R^2) along with reductions in MAE, RMSE, and MAPE

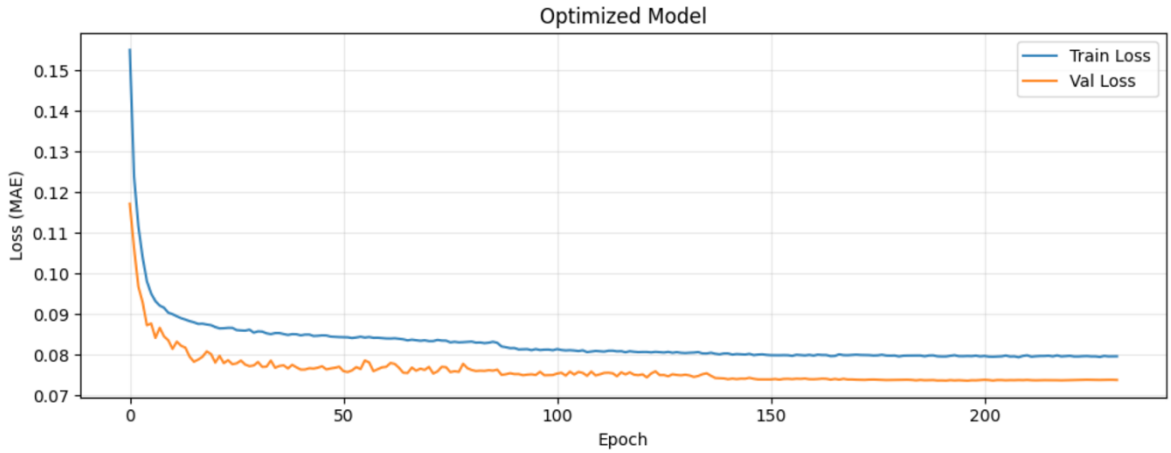
• The final model demonstrates a well-balanced trade-off between model complexity and performance

7.4. Training Behavior After Optimization

The optimized model's learning curve shows:

- Smooth and consistent convergence across training epochs
- Slight but controlled fluctuations in validation loss due to model flexibility
- Minimal gap between training and validation loss, indicating good generalization
- No significant overfitting observed

These observations confirm that the applied optimization techniques successfully improved model stability, generalization, and predictive reliability.



8. Challenges and Solutions

During the development of the energy consumption forecasting model, several challenges were encountered related to time-series modeling, data handling, and model performance. The following outlines the key issues and how they were addressed.

1. Data Leakage Risk

Challenge:
Time-series datasets are highly sensitive to data leakage, especially when performing scaling, feature selection, and validation splitting.

Solution:

- A chronological train-test split was used instead of random splitting
- Scaling (MinMaxScaler) was fitted only on training data
- Feature selection and preprocessing steps were strictly applied without using future data

This ensured realistic and unbiased model evaluation.

2. Poor Performance of Complex Models

Challenge:
Some architectures (e.g., LSTM_Model_C and GRU_Model_D) showed relatively poor performance, with higher MAE and lower R^2 values mainly due to excessive complexity and unstable learning behavior.

Solution:

- Reduced model complexity (fewer layers and units)
- Focused on simpler and more stable architectures (LSTM_Model_B)
- Applied dropout regularization to improve generalization

This demonstrated that moderate model complexity performs better than overly complex architectures.

3. Overfitting in Deep Learning Models

Challenge:
Initial training showed signs of overfitting, where training loss decreased but validation loss plateaued or fluctuated.

Solution:

- Applied Dropout layers (0.1- 0.3)

- Used EarlyStopping to stop training when validation loss stopped improving
- Implemented ReduceLROnPlateau to adjust learning rate dynamically

These techniques helped achieve smoother convergence and reduced overfitting.

4. Hyperparameter Selection Complexity

Challenge:
Selecting optimal hyperparameters (units, layers, learning rate, dropout, batch size) manually is inefficient and may lead to suboptimal performance.

Solution:
A randomized hyperparameter search was implemented:

- Explored multiple combinations efficiently
- Selected best configuration based on validation MAE
- Included a baseline configuration to ensure stability

This systematic approach improved model performance without excessive computation.

5. Computational Cost

Challenge:
Deep learning models, especially during hyperparameter tuning, required significant training time.

Solution:

- Limited search space (units capped at 128, layers ≤ 2)
- Used random sampling instead of full grid search
- Used early stopping to reduce unnecessary epochs

This balanced performance improvement with computational efficiency.

9. Conclusion

This assessment developed and evaluated multiple machine learning and deep learning models for predicting appliance energy consumption using time-series data. Baseline models, including Linear Regression and Random Forest, were first implemented to establish reference performance. Among these, Linear Regression achieved relatively strong results, while Random Forest performed poorly due to its limited ability to capture temporal dependencies.

Deep learning models, including LSTM and GRU architectures, were then introduced to better model sequential patterns in the data. Among the evaluated models, LSTM_Model_B achieved the best performance, demonstrating lower error metrics (MAE, RMSE, MAPE) and higher R^2 compared to other architectures. This confirms that sequence-based models are more effective for time-series forecasting tasks.

Further improvements were achieved through systematic optimization techniques, including hyperparameter tuning, dropout regularization, early stopping, and learning rate scheduling. The final optimized model demonstrated consistent improvements across all evaluation metrics, achieving an MAE of approximately 12.04 Wh and an R^2 of approximately 0.7, indicating strong predictive capability and improved generalization.

Overall, the results show that deep learning models, particularly LSTM-based architectures, outperform traditional machine learning approaches when modeling temporal dependencies. However, the models still face challenges in accurately predicting sudden spikes in energy consumption.

This assessment highlights the importance of balancing model complexity, optimization techniques, and temporal feature engineering in achieving reliable time-series forecasting performance.

For future work, the model performance can be further optimized through more extensive hyperparameter tuning and improved feature engineering. Increasing the dataset size and diversity may also enhance model generalization. Finally, deploying the model in a real-time prediction system and evaluating its performance in practical scenarios would provide further insights into its robustness and applicability.

10. References

Time Series Forecasting with LSTM Neural Networks (TensorFlow Tutorial)

Kaggle - Time Series Feature Engineering

Time series models for forecasting